

面向 LLM / Agent Systems 科研的深度学习零基础课

第 01 讲

从一个数字到 Tensor: Scalar、Vector、Matrix、Tensor 与 Shape

本讲目标：把“张量”从一个听起来抽象的名词，变成你能看见、能手画、能预测 Shape、能用 NumPy 验证的数据结构。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

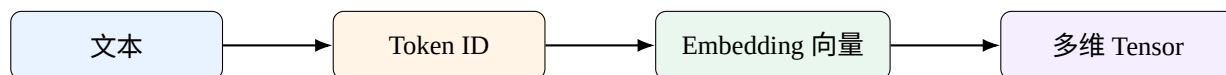
1 为什么第一讲从“数据长什么样”开始	4
1.1 本讲学完后你必须能做什么	4
2 先从最简单的数字开始：Scalar 标量	4
2.1 【原理】Scalar 是什么	4
2.2 【通俗解释】Scalar 就是一张纸上的一个格子	5
2.3 Scalar 的 Shape 为什么是空的	5
3 Vector 向量：一排有顺序的数字	6
3.1 【原理】Vector 是什么	6
3.2 【通俗解释】Vector 是一排储物格	6
3.3 Vector 里的每个数字可以代表什么	6
4 Matrix 矩阵：把多个向量排成二维表	7
4.1 【原理】Matrix 是什么	7
4.2 【图解】两个轴到底在哪里	8
4.3 【具体数字例子】3 个 Token，每个 Token 4 个特征	8
5 Tensor 张量：把二维继续推广	8
5.1 【原理】Tensor 是什么	8
5.2 3D Tensor：一摞矩阵	9
5.3 为什么深度学习特别依赖 Tensor	9
6 Dimension、Axis、Shape：最容易混淆的三个词	9
6.1 Dimension / ndim：有多少个轴	9
6.2 Axis：具体是哪一个方向	10
6.3 Shape：把每个轴的长度按顺序写出来	10
6.4 Size：一共有多少个数字	10
7 为什么 Shape 在深度学习里如此重要	11
8 一组数字为什么可以有不同 Shape	11
9 NumPy：第一阶段的“显微镜”	12
9.1 创建 Scalar、Vector、Matrix、Tensor	12

9.2 为什么 (3,) 后面有逗号	13
10 【代码实验】用 NumPy 亲眼看 Shape	13
10.1 实验问题	13
10.2 完整核心代码	13
10.3 你运行代码前应该先预测	14
11 【常见误区】这一讲最容易错在哪里	14
11.1 误区 1: Tensor 就一定是 3D 或更高维	14
11.2 误区 2: 矩阵的“2 维”表示里面只有两个数字	15
11.3 误区 3: Embedding 是一个很神秘的新数据类型	15
11.4 误区 4: Shape 相同, 语义就相同	15
11.5 误区 5: 所谓“768 维向量”等于 768D Tensor	15
12 与以前 LLM 课程的重新连接	15
12.1 Token Embedding	15
12.2 Q、K、V	15
12.3 Transformer Block	16
13 科研视角: 为什么从第一讲就训练 Shape 意识	16
14 本讲最小必要公式	16
15 本讲小结: 把四个对象放到同一张地图里	17
16 本讲练习入口	17
17 下一讲预告	18

1 为什么第一讲从“数据长什么样”开始

深度学习看起来有很多复杂名词：神经元、权重、Loss、Gradient、Attention、Transformer。可是从计算机真正执行的角度看，它们最终都落到一件很朴素的事情上：**对一堆按照特定形状组织起来的数字做运算。**

比如一个大语言模型收到一句话以后，不会在计算机里直接处理“我喜欢人工智能”这几个汉字。它会经历一系列表示变化：



你以前在 LLM 课程里已经见过 Token、Embedding、Attention、Transformer，但这些概念背后的计算都依赖今天要学的四个对象：

Scalar (标量) → Vector (向量) → Matrix (矩阵) → Tensor (张量)

本讲最重要的一句话

Tensor 并不是某种神秘的“AI 数据”。你可以先把它理解为：**按照一个或多个轴组织起来的一批数字**。Scalar、Vector、Matrix 都可以看成 Tensor 的特殊情况。

1.1 本讲学完后你必须能做什么

学完以后，你至少应该能做到下面六件事：

1. 看到一个数字、一个列表、一个二维表，判断它更接近 Scalar、Vector 还是 Matrix；
2. 解释 Tensor 是什么，而不是只会说“多维数组”；
3. 看懂 (3,)、(2, 3)、(4, 10, 768) 这类 Shape；
4. 区分 **dimension / rank** 与某个轴的 **size**；
5. 用 NumPy 创建这些对象，并检查 ndim、shape、size；
6. 能把 LLM 中常见的 [batch, sequence, hidden] 与今天的 Tensor 结构连接起来。

2 先从最简单的数字开始：Scalar 标量

2.1 【原理】Scalar 是什么

Scalar (标量) 就是一个单独的数值。它没有“横着排几个”“竖着排几行”这样的结构。

例如：

3, -1.5, 0, 98.6

它们都可以是 Scalar。

在深度学习里，一个 Scalar 可能表示：

- 某一个神经元的输入值；
- 某一个权重；
- 某一个样本的 Loss；
- 当前 Learning Rate；
- 某一步的概率值；
- 某个梯度分量。

2.2 【通俗解释】Scalar 就是一张纸上的一个格子

想象你拿出一张方格纸，只在一个格子里写：

3.5

这就是最简单的数据对象。没有第 2 个数字，也没有行列关系。

具体数字例子：一个 Loss

假设模型对一个样本预测以后得到 Loss：

$$L = 0.73$$

这里的 L 就是一个 Scalar。它虽然由很多计算得到，但最终只是一个数字：0.73。

2.3 Scalar 的 Shape 为什么是空的

在 NumPy 中：

```
import numpy as np

x = np.array(3.5)
print(x)
print(x.ndim)
print(x.shape)
```

你会看到：

```
3.5
0
()
```

这里：

- `ndim = 0`：它没有轴；
- `shape = ()`：没有任何轴的长度需要描述。

常见误区 / 风险

“0 维”并不是说它没有数据。它有一个数值，只是没有可以沿着某个轴继续索引的结构。

3 Vector 向量：一排有顺序的数字

3.1 【原理】Vector 是什么

Vector（向量） 可以先理解为：一组按一个轴排列的数字。

例如：

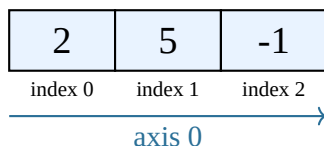
$$\mathbf{x} = [2, 5, -1]$$

它包含 3 个数字，因此我们可以说这个向量的长度是 3。

如果用 NumPy 表示：

```
x = np.array([2, 5, -1])
print(x.ndim)    # 1
print(x.shape)   # (3,)
```

3.2 【通俗解释】Vector 是一排储物格



只有一个方向可以移动，所以它是 1 维对象。

3.3 Vector 里的每个数字可以代表什么

这取决于语境。比如一个学生样本：

`[172, 65, 20]`

可以约定为：

- 第 1 个数字：身高 cm；
- 第 2 个数字：体重 kg；
- 第 3 个数字：年龄。

同一个 Vector 的三个位置表达三个 Feature（特征）。

在深度学习里，向量非常常见：

- 一个 Token 的 Embedding；
- 一个隐藏状态 hidden state；
- 一个样本的特征；
- 一个神经元组的输出；
- 一组 logits；
- 一组梯度。

与 LLM / Transformer / Agent Systems 的连接

你以前见过 Token Embedding。假设隐藏维度为 4，一个 Token 可能被表示成：

$$[0.12, -0.37, 0.91, 0.08]$$

这就是一个长度为 4 的向量。真实 LLM 里长度可能是几千，而不是 4，但数学对象并没有变。

4 Matrix 矩阵：把多个向量排成二维表

4.1 【原理】Matrix 是什么

Matrix（矩阵） 是一组按照两个轴排列的数字。最常见的说法是“行”和“列”。

例如：

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

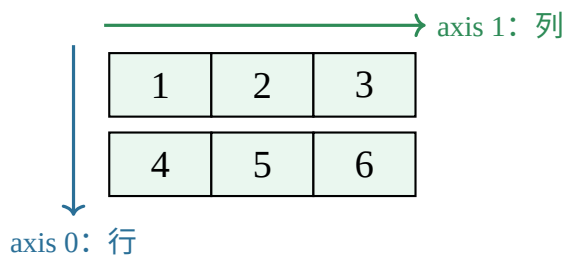
它有：

- 2 行；
- 3 列；
- 总共 6 个数字。

因此 Shape 是：

$$(2, 3)$$

4.2 【图解】两个轴到底在哪里



Shape 的读法

(2, 3) 不是“第一个维度叫 2，第二个维度叫 3”。更准确地说：这个对象有两个轴；axis 0 的长度是 2，axis 1 的长度是 3。

4.3 【具体数字例子】3 个 Token，每个 Token 4 个特征

假设一句极短文本经过 Embedding 后有 3 个 Token，每个 Token 的向量长度为 4：

$$X = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix}$$

那么：

$$X.shape = (3, 4)$$

这里可以约定：

- axis 0: Token 位置，共 3 个；
- axis 1: Embedding / hidden 维度，共 4 个。

注意：同一个 (3, 4)，在别的任务里也可能表达别的含义。**Shape 只告诉你结构，不自动告诉你语义。**

5 Tensor 张量：把二维继续推广

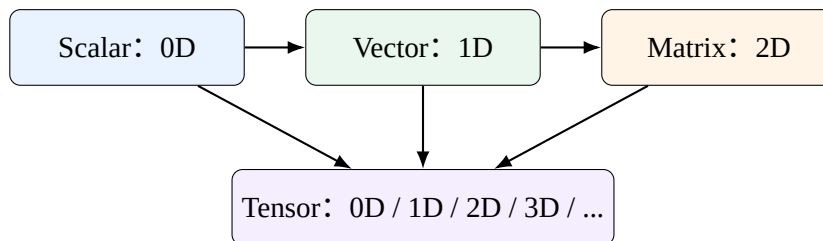
5.1 【原理】Tensor 是什么

在现代深度学习工程里，可以采用一个非常实用的定义：

核心概念

Tensor (张量) 是一个可以拥有 0 个、1 个、2 个或更多轴的数值数组。Scalar、Vector、Matrix 可以分别视为 0 维、1 维、2 维 Tensor。

因此：



5.2 3D Tensor：一摞矩阵

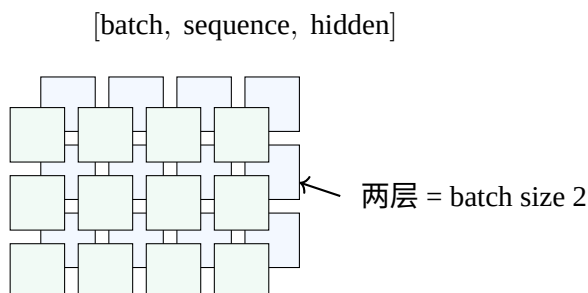
假设我们一次处理 2 个句子，每个句子有 3 个 Token，每个 Token 用长度 4 的向量表示，那么数据的 Shape 可以写成：

$(2, 3, 4)$

你可以把它读成：

2 个样本 × 每个样本 3 个 Token × 每个 Token 4 个数

也就是：



每一层：3 个 Token × 4 个 hidden 数值

5.3 为什么深度学习特别依赖 Tensor

神经网络需要同时处理大量样本和大量特征。如果每一个数字都单独处理，计算会极其低效。Tensor 允许我们把很多数字组织成规则的形状，然后让 CPU/GPU 对整块数据执行高度并行的运算。

从底层视角看，Tensor 解决两个问题：

1. **结构问题**：这些数字怎样组织？哪些数字属于同一个样本、Token、通道或特征？
2. **计算问题**：怎样一次性对大量数字执行矩阵乘法、加法、归一化等运算？

6 Dimension、Axis、Shape：最容易混淆的三个词

6.1 Dimension / ndim：有多少个轴

对于：

$A.shape = (2, 3, 4)$

我们说它有 3 个轴，因此：

$$A.ndim = 3$$

在工程语境中，这常被称为 3D Tensor 或 rank-3 tensor。

“维度”这个词有两个常见用法

“dimension” 在不同语境里容易引起歧义。比如 “768-dimensional embedding” 常常表示一个向量有 768 个分量，即 shape 为 (768,)；但这个向量作为数组其实只有 1 个轴，ndim=1。因此读论文和代码时要根据上下文区分 “轴的个数” 和 “某个表示空间的宽度”。

6.2 Axis：具体是哪一个方向

如果：

$$X.shape = (2, 3, 4)$$

则：

- axis 0 的 size = 2；
- axis 1 的 size = 3；
- axis 2 的 size = 4。

在 LLM 中，一个典型语义可能是：

- axis 0：batch；
- axis 1：sequence / token position；
- axis 2：hidden dimension。

6.3 Shape：把每个轴的长度按顺序写出来

必要公式

如果一个 Tensor 有 n 个轴，每个轴的长度分别为 $d_0, d_1, \dots, d_{n-1}$ ，那么可以把它的 Shape 写成：

$$(d_0, d_1, \dots, d_{n-1})$$

其中 n 是轴的个数，而每个 d_i 是对应轴的 size。

6.4 Size：一共有多少个数字

对于 shape：

$$(2, 3, 4)$$

总元素数量是：

$$2 \times 3 \times 4 = 24$$

NumPy 中：

```
print(x.shape) # (2, 3, 4)
print(x.ndim)  # 3
print(x.size)  # 24
```

本节检查点

看到 (8, 16, 64)，你现在应该立刻能回答：这是 3 维 Tensor；三个轴长度分别是 8、16、64；总元素数为 $8 \times 16 \times 64 = 8192$ 。至于三个轴分别表示 batch、token 还是别的含义，需要看任务语境。

7 为什么 Shape 在深度学习里如此重要

很多初学者把 Shape 当成“调试代码时偶尔打印一下的东西”。实际上，Shape 是理解神经网络计算的核心语言。

后面你会学到：

$$y = Wx + b$$

如果你不知道 x, W, b 的 Shape，就无法判断乘法是否成立，也无法知道输出是什么。

Transformer 中更明显。以后会反复看到：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

以及：

$$QK^T$$

这些公式的第一检查方法不是先看符号有多复杂，而是问：

每个 Tensor 的 Shape 是什么？下一步 Shape 能不能对上？

科研视角

读深度学习论文时，很多公式错误、实现错误、复现错误，最终都能暴露为 Shape 或 axis 语义不一致。养成“公式旁边写 Shape”的习惯，是以后读 Transformer 与 Agent Systems 底层模型实现时非常实用的能力。

8 一组数字为什么可以有不同 Shape

这是一个很重要但经常被忽略的问题。

假设我们有 6 个数字：

$$1, 2, 3, 4, 5, 6$$

它们可以组织成：

$$[1, 2, 3, 4, 5, 6] \quad \text{shape} = (6,)$$

也可以组织成：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \text{shape} = (2, 3)$$

还可以组织成：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad \text{shape} = (3, 2)$$

甚至可以组织成 shape 为 (1, 2, 3) 的 3D Tensor。

核心概念

元素相同，不代表数据结构相同。 Shape 会影响后续哪些运算合法、沿哪个方向做聚合、矩阵乘法怎样对齐，以及每个轴在任务里代表什么。

这也是本讲实验要验证的核心现象。

9 NumPy：第一阶段的“显微镜”

我们暂时不急着使用 PyTorch。原因不是 PyTorch 难，而是它会自动替你做太多事情。第一阶段使用 NumPy，目的是让数据结构和运算尽量透明。

9.1 创建 Scalar、Vector、Matrix、Tensor

```
import numpy as np

# 0D Tensor / Scalar
scalar = np.array(7.5)

# 1D Tensor / Vector
vector = np.array([1.0, 2.0, 3.0])

# 2D Tensor / Matrix
matrix = np.array([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
])

# 3D Tensor
```

```
three_d = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])
```

对应信息：

对象	ndim	shape	size
scalar	0	()	1
vector	1	(3,)	3
matrix	2	(2, 3)	6
three_d	3	(2, 2, 3)	12

9.2 为什么 (3,) 后面有逗号

在 Python 里，(3,) 表示只有一个元素的 tuple（元组）。如果写成 (3)，Python 只会把它当作数字 3 外面加了括号。

所以：

- (3,)：1 个轴，长度为 3；
- (1,3)：2 个轴，长度分别为 1 和 3；
- 它们不是同一个 Shape。

10 【代码实验】用 NumPy 亲眼看 Shape

本讲代码文件会完整提供。这里先理解代码在做什么。

10.1 实验问题

我们要验证三个问题：

1. Scalar、Vector、Matrix、3D Tensor 的 ndim、shape、size 有什么规律？
2. 同样 6 个数字改变 Shape 后，总元素数是否变化？
3. Shape 改变以后，索引方式和“轴”的含义是否随之改变？

10.2 完整核心代码

```
import numpy as np

# 创建四种不同结构的数据
```

```
scalar = np.array(7.5)
vector = np.array([1, 2, 3])
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
tensor3d = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])

# 把对象放在列表里，便于统一观察
objects = [
    ("scalar", scalar),
    ("vector", vector),
    ("matrix", matrix),
    ("tensor3d", tensor3d)
]

# 逐个打印数据结构信息
for name, x in objects:
    print("=" * 50)
    print("name :", name)
    print("value:\n", x)
    print("ndim :", x.ndim)
    print("shape:", x.shape)
    print("size :", x.size)
```

10.3 你运行代码前应该先预测

不要先运行。先在纸上填写：

对象	预测 ndim	预测 shape	预测 size
scalar			
vector			
matrix			
tensor3d			

然后再运行程序。这种“先预测、后验证”的习惯以后会延续到矩阵乘法、Backpropagation 和 Transformer。

11 【常见误区】这一讲最容易错在哪里

11.1 误区 1：Tensor 就一定是 3D 或更高维

不对。在深度学习框架语境里，一个 0D Scalar 也可以是 Tensor，一个 1D Vector 也可以是 Tensor。

11.2 误区 2：矩阵的“2 维”表示里面只有两个数字

不对。“2 维”说的是有两个轴，不是只有两个元素。一个 $(1000, 1000)$ 的矩阵仍然是 2D，但有一百万个元素。

11.3 误区 3：Embedding 是一个很神秘的新数据类型

不是。一个 Token Embedding 通常就是一个浮点向量。多个 Token 排在一起形成矩阵；多个句子组成 batch 后进一步形成 3D Tensor。

11.4 误区 4：Shape 相同，语义就相同

不对。两个 Tensor 都可能是 $(32, 128, 768)$ ，一个可能表示文本隐藏状态，另一个也可能来自完全不同的任务。Shape 是结构信息，不足以表达全部语义。

11.5 误区 5：所谓“768 维向量”等于 768D Tensor

通常不这样理解。长度 768 的向量，其 NumPy Shape 是 $(768,)$ ，数组的 ndim 是 1；“768 维”说的是向量空间中有 768 个分量。

12 与以前 LLM 课程的重新连接

12.1 Token Embedding

以前你知道 Token 会变成 Embedding。现在可以更精确地说：

假设：

- batch size = 2；
- 每个样本 sequence length = 4；
- hidden size = 3。

那么 Embedding 输出可以是：

$$X.shape = (2, 4, 3)$$

含义：

2 个句子 × 每句 4 个 Token × 每个 Token 3 个数

真实模型中 hidden size 可能是 768、4096、8192 等，但结构思想一样。

12.2 Q、K、V

你以前见过：

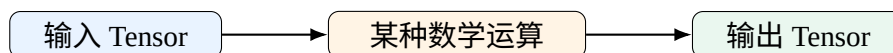
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

今天还不学矩阵乘法，但先建立一个认识：**这里的 X, W_Q, Q 都不是抽象符号，它们都是有具体 Shape 的 Tensor。**

第 4 讲开始，你会手算这些矩阵乘法的底层版本。

12.3 Transformer Block

以后看到模型结构图，不要把方框理解成“模块名称”。更底层地看，每一个方框都在做：



后面的 59 讲，本质上就是逐步解释这些“某种数学运算”到底是什么。

13 科研视角：为什么从第一讲就训练 Shape 意识

你未来的主方向是 Agent Systems，但 Agent System 的认知核心通常依赖 LLM 或其他神经模型。即使你不研究基础模型训练，也会遇到论文中的：

- hidden size;
- embedding dimension;
- batch size;
- context length;
- logits;
- attention map;
- activation tensor;
- feature tensor;
- model output shape。

科研视角

科研中遇到一段陌生模型代码时，一个非常有效的最低成本方法是：在关键节点打印 Tensor 的 shape、dtype 和少量数值样本。你不需要先完全理解整个模型，就可以先建立“数据从哪里来、以什么形状流到哪里”的计算路径。

这也是以后做复现实验、读开源实现和定位 Bug 时的最低闭环能力之一。

14 本讲最小必要公式

这一讲没有复杂数学，但要形成几个记号习惯。

必要公式

向量：

$$\mathbf{x} = [x_1, x_2, \dots, x_d]$$

如果包含 d 个分量，可以写作 $\mathbf{x} \in \mathbb{R}^d$ 。当前只需把 $\mathbb{R}^d$ 理解为“由 d 个实数组成的向量空间”。

矩阵：

$$A \in \mathbb{R}^{m \times n}$$

表示 A 有 m 行、 n 列。

3D Tensor：

$$X \in \mathbb{R}^{B \times T \times D}$$

在 LLM 中常可解释为 batch size B 、sequence length T 、hidden size D 。

注意：这里的 $\mathbb{R}$ 和上标不是为了增加难度，而是为了让你以后读论文时看到这种符号不会停住。

15 本讲小结：把四个对象放到同一张地图里

对象	直观形态	ndim	示例 Shape	深度学习例子
Scalar	一个数	0	()	单个 Loss、单个权重
Vector	一排数	1	(768 ,)	一个 Token Embedding
Matrix	二维表	2	(128 , 768)	一段序列的 hidden states
Tensor	多轴数值数组	0+	(32 , 128 , 768)	一个 batch 的 hidden states

第 01 讲真正需要记住的，不是定义，而是这个思维方式

以后看到任何深度学习对象，先问四个问题：

- 1. 它里面装的是什么数字？
- 2. 它有几个轴？
- 3. 每个轴的长度是多少？
- 4. 每个轴在这个任务里分别代表什么？

这四问会一路用到矩阵乘法、MLP、Attention 和 Transformer。

16 本讲练习入口

完整练习和答案已经放在单独的《第 01 讲练习题与答案》中。这里先做 4 个口头检查：

1. $(5,)$ 是几维 Tensor? 一共有几个元素?
2. $(2,3)$ 是几维 Tensor? 一共有几个元素?
3. $(4,10,768)$ 中, `ndim` 和 `size` 分别是多少?
4. 如果一个 LLM hidden state 的 shape 是 $(8,128,4096)$, 你能给三个轴提供一种合理的语义解释吗?

如果这四个问题能够不看答案说清楚, 就达到了本讲最低掌握线。

17 下一讲预告

下一讲进入: **Shape、索引、切片、转置与维度变化。**

今天我们只解决“数据是什么形状”。下一讲开始研究:

- 怎样从 Tensor 中拿出某一个元素;
- 怎样取出一行、一列、一段;
- `reshape` 到底改变了什么;
- `transpose` 为什么会改变轴的顺序;
- `squeeze` / `unsqueeze` 为什么在深度学习代码里频繁出现;
- 为什么很多 Bug 本质上是 Shape Bug。